TUGAS JOBSHEET 7 - PERULANGAN 1

Praktikum Dasar Pemrograman

NAMA	**Nasrul Ardhiansyah**
NIM	**264107020157**
KELAS	**TI 1C**
NO. ABSEN	**24**

Pertanyaan Percobaan 1

1. Sebutkan dan tunjukkan masing-masing komponen perulangan FOR pada kode program Percobaan 1!

Jawab:

Struktur dasar perulangan for terdiri dari empat komponen utama:

- **Inisialisasi (in t i = 1):** Inisialisasi awal variabel pencacah (counter) yang dieksekusi tepat satu kali ketika perulangan pertama kali dimulai. Variabel counter i diatur dengan nilai awal 1.
- **Kondisi (i <= 10):** Syarat kelanjutan perulangan berupa ekspresi boolean. Perulangan akan terus mengeksekusi blok kode selama nilai i bernilai kurang dari atau sama dengan 10. Begitu i mencapai nilai 11, kondisi bernilai false dan perulangan berhenti.
- **Update / Step (i++):** Tahap pembaruan nilai counter setiap kali satu siklus iterasi selesai dijalankan. Sintaks ini menaikkan nilai i sebesar 1 (inkremen).
- **Badan Perulangan (Body Loop):** Seluruh ba ris instruksi di dalam kurung kurawal { ... } yang dieksekusi berulang kali sebanyak 10 putaran, yaitu proses membaca input nilai mahasiswa dari keyboard dan melakukan pengecekan nilai tertinggi serta terendah.

2. Mengapa variabel tertinggi diinisialisasi 0 dan terendah diinisialisasi 100? Apa yang terjadi jika variabel tertinggi diinisialisasi 100 dan terendah diinisialisasi 0?

Jawab:

- **Alasan Inisialisasi Awal:** Skala nilai kuis mahasiswa umumnya berkisar dari 0 hingga 100. Variabel tertinggi diinisialisasi dengan angka paling minimum yang mungkin didapatkan (0) agar input nilai pertama dari mahasiswa manapun (yang bernilai >= 0) langsung berhasil menggantikan nilai awal tersebut jika lebih besar. Sebaliknya, variabel terendah diinisialisasi dengan angka paling maksimum (100) agar input nilai pertama langsung berhasil menggantikan nilai awal tersebut jika lebih kecil.

• **Dampak Jika Posisi Dibalik:** Jika tertinggi diinisialisasi 100, maka kondisi percabangan `if (nilai > tertinggi)` tidak akan pernah bernilai true karena tidak ada input nilai mahasiswa yang melampaui 100. Akibatnya nilai tertinggi akan salah dan tetap bernilai 100. Sebaliknya, jika ter endah diinisialisasi 0, maka kondisi `if (nilai < terendah)` tidak akan pernah terpenuhi karena tidak ada nilai mahasiswa yang lebih kecil dari 0. Akibatnya variabel terendah akan salah dan tetap bernilai 0.

3. Jelaskan fungsi dan alur kerja dari potongan kode berikut!

```
if (nilai > tertinggi) {
    tertinggi = nilai;
}
if (nilai < terendah) {
    terendah = nilai;
}
```

Jawab:

• **Fungsi:** Berfungsi untuk mengevaluasi data nilai yang baru saja dimasukkan terhadap rekor nilai ekstrem (maksimum dan minimum) saat ini, kemudian memperbarui variabel tertinggi atau terendah jika nilai baru tersebut memecahkan rekor sebelumnya.

• **Alur Kerja:** (1) Program mula-mula mengecek kondisi pertama: `if (nilai > tertinggi)`. Jika nilai baru lebih besar daripada rekor tertinggi yang tercatat, maka nilai variabel tertinggi diganti dengan nilai baru tersebut (`terting gi = nilai;`). (2) Selanjutnya, program mengecek kondisi kedua secara terpisah: `if (nilai < terendah)`

4. Modifikasi kode program sehingga terdapat perhitungan untuk menentukan berapa mahasiswa yang lulus dan yang tidak lulus berdasarkan batas kelulusan (nilai minimal 60). Tampilkan jumlah mahasiswa lulus dan tidak lulus setelah menampilkan nilai tertinggi dan terendah!

Jawab:

Pada program dimodifikasi dengan menambahkan dua variabel pencacah akumulatif, yaitu `int jmlLulus = 0, jmlTidakLulus = 0;`. Di dalam perulangan for, ditambahkan seleksi `if (nilai >= 60) { jmlLulus++; } else { jmlTidakLulus++; }`. Setelah loop selesai, kedua hasil tersebut ditampilkan ke layar.

Kode Program Modifikasi: SiakadForModif24.java

Pertanyaan Percobaan 2

1. Pada potongan kode berikut, tentukan maksud dan kegunaan dari sintaks berikut:
- nilai < 0 || nilai > 100
- continue

```
if (nilai < 0 || nilai > 100) {
    System.out.println(x:"Nilai tidak valid. Masukkan lagi nilai yang valid!");
    continue;
}
```

Jawab:

• **nilai < 0 || nilai > 100:** Maksudnya adalah ekspresi relasional berkondisi ganda dengan operator OR (||) untuk memeriksa apakah nilai berada di luar batas akademis yang sah (kurang dari 0 ATAU lebih dari 100). Kegunaannya sebagai gerbang validasi input (data validation) agar data tidak valid tidak diproses lebih jauh.

• **continue:** Maksudnya adalah perintah jump statement dalam struktur perulangan. Kegunaannya adalah untuk segera mengabaikan (melewati) seluruh baris kode yang tersisa di bawahnya pada iterasi saat ini (termasuk pembaruan i++), lalu langsung melompat kembali ke awal perulangan untuk mengecek kondisi `while (i < jml)`. Efeknya, counter i tidak bertambah, sehingga mahasiswa dengan nomor urut yang sama diminta memasukkan ulang nilai yang valid.

2. Mengapa sintaks i++ dituliskan di akhir perulangan WHILE? Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?

Jawab:

• **Alasan Ditulis di Akhir:** Sintaks i++ dituliskan di akhir agar kenaikan nomor indeks mahasiswa hanya terjadi jika input nilai telah dinyatakan valid dan selesai dikonversi ke nilai huruf. Jika nilai tidak valid, perintah continue akan melompati i++, sehingga indeks i tetap sama untuk input pengganti berikutnya.

• **Dampak Jika Ditulis di Awal:** (1) Penomoran mahasiswa akan langsung meloncat bertambah 1 sebelum input dibaca , sehingga pada mahasiswa pertama prompt langsung mencetak mahasiswa ke-2; (2) Logika validasi rusak: jika pengguna memasukkan nilai tidak valid dan continue dieksekusi,

3. Apabila jumlah mahasiswa yang dimasukkan adalah 19, berapa kali perulangan WHILE akan berjalan?

Jawab:

• **Jika Semua Input Valid (0 - 100):** Perulangan WHILE akan berjalan tepat sebanyak 19 kali, dimulai dari indeks i = 0 hingga i = 18. Pada saat i = 19, kondisi evaluasi `while (i < 19)` bernilai false dan perulangan berakhir.

• **Jika Terdapat Input Tidak Valid:** Setiap kali pengguna memasukkan nilai yang keliru (< 0 atau > 10 0), pernyataan continue dieksekusi tanpa menaikkan i. Oleh karena itu, jumlah perulangan total adalah 19 kali ditambah jumlah kesalahan input yang dilakukan pengguna (misal terjadi 3 kali kesalahan input, maka perulangan berjalan sebanyak 19 + 3 = 22 kali).

4. Modifikasi kode program sehingga apabila terdapat mahasiswa yang mendapat nilai A, program menampilkan pesan tambahan "Bagus, pertahankan nilainya"! Commit dan push kode program ke Github.

Jawab:

Modifikasi dilakukan pada blok kondisi nilai A dengan menambahkan satu baris instruksi `System.out.println("Bagus, pertahankan nilainya!");`.

Pertanyaan Percobaan 3

1. Pada penggunaan DO-WHILE ini, apabila nama pelanggan yang dimasukkan pertama kali adalah “batal”, maka berapa kali perulangan dilakukan?

Jawab:

Perulangan dilakukan tepat 1 kali. Sifat dari perulangan do-while adalah post-test loop (exit-controlled), yang menjamin blok pernyataan di dalam blok do pasti dijalankan minimal satu kali sebelum kondisi dievaluasi. Ketika pertama kali dijalankan, pengguna memasukkan teks "batal", blok percabangan `if (namaPelanggan.equalsIgnoreCase("batal"))` bernilai true, program mencetak "Transaksi dibatalkan.", lalu perintah `break;` seketika menghentikan eksekusi loop.

2. Sebutkan kondisi berhenti yang digunakan pada perulangan DO-WHILE tersebut!

Jawab:

Kondisi berhenti pada perulangan tersebut bukan diatur pada parameter while(true), melainkan dikendalikan oleh struktur pemilihan di dalam badan loop: `if (namaPelanggan.equalsIgnoreCase("batal"))` yang diikuti perintah `break;`.

3. Apa fungsi dari penggunaan nilai true pada kondisi DO-WHILE?

Jawab:

Penggunaan nilai `true` pada `while (true)` berfungsi untuk membentuk perulangan tak hingga (infinite loop) yang disengaja. Tujuannya adalah agar kasir dapat terus melayani transaksi demi transaksi pelanggan secara dinamis ta npa dibatasi kuota putaran di awal, sampai kasir memutuskan keluar dengan memasukkan kata sentinel ("batal") yang memicu `break;`.

4. Mengapa perulangan DO-WHILE tersebut tetap berjalan meskipun tidak ada komponen inisialisasi dan update?

Jawab:

Komponen inisialisasi dan update han ya wajib pada perulangan berbasis pencacah bilangan (counter-controlled loop) seperti `for` atau `while` berhitung. Pada kasus ini, program menerapkan perulangan berbasis sentinel / event-driven loop yang kondisinya bernilai konstan true.

Tugas Praktikum

Tugas 1: Program Penjualan Tiket Bioskop Harian

Ketentuan Program:

• **Harga Tiket:** Tiket dijual dengan harga Rp 50.000 per tiket.

• **Ketentuan Diskon:** (1) Pembelian > 4 tiket mendapatkan diskon 10%; (2) Pembelian > 10 tiket mendapatkan diskon 15%; (3) Pembelian <= 4 tiket tidak mendapat diskon.

• **Validasi Input:** Jika masukan jumlah tiket bernilai negatif, program mengabaikan input tersebut, memunculkan peringatan, dan meminta input ulang menggunakan `continue;`.

• **Penghentian & Rekapitulasi:** Masukkan nilai 0 untuk mengakhiri transaksi harian (`break;`), lalu program mencetak rekapitulasi total tiket terjual dan total uang hasil penjualan selama satu hari.

```java
import java.util.Scanner;

public class Bioskop24 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final int HARGA_TIKET = 50000;
        int totalTiketTerjual = 0;
        double totalPenjualan = 0;
        int jumlahTiket;

        System.out.println("=== PROGRAM PENJUALAN TIKET BIOSKOP ===");
        System.out.println("Harga per tiket: Rp 50.000");
        System.out.println("Ketik '0' jika transaksi hari ini sudah selesai.\n");

        while (true) {
            System.out.print("Masukkan jumlah tiket yang dibeli (0 untuk selesai): ");
            jumlahTiket = sc.nextInt();

            if (jumlahTiket == 0) {
                System.out.println("Sesi penjualan tiket hari ini diakhiri.");
                break;
            }

            if (jumlahTiket < 0) {
                System.out.println("Jumlah tiket tidak valid (tidak boleh negatif)! Masukkan ulang.");
                continue;
            }

            double diskon = 0;
            if (jumlahTiket > 10) {
                diskon = 0.15; // Diskon 15%
            } else if (jumlahTiket > 4) {
                diskon = 0.10; // Diskon 10%
            }

            double totalHargaTransaksi = jumlahTiket * HARGA_TIKET;
            double nominalDiskon = totalHargaTransaksi * diskon;
            double bayar = totalHargaTransaksi - nominalDiskon;

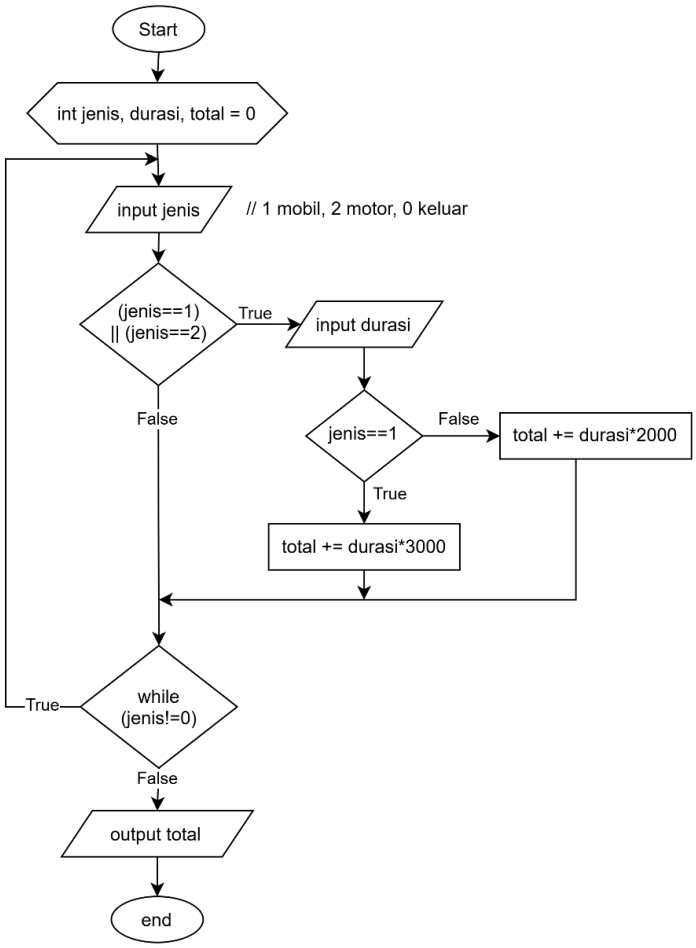
            totalTiketTerjual += jumlahTiket;
            totalPenjualan += bayar;

            System.out.println("Diskon: " + (int)(diskon * 100) + "% (Rp " + (long)nominalDiskon + ")");
            System.out.println("Total bayar transaksi: Rp " + (long)bayar);
            System.out.println("-----------------------------------------------");
        }

        System.out.println("\n========== REKAPITULASI PENJUALAN HARIAN ==========");
        System.out.println("Total tiket terjual        : " + totalTiketTerjual + " tiket");
        System.out.println("Total harga penjualan tiket: Rp " + (long)totalPenjualan);
        System.out.println("================================================");
        sc.close();
    }
}
```

Tugas 2: Program Pembayaran Parkir (Implementasi Flowchart)

Program dirancang untuk menghitung total pembayaran parkir dari beberapa kendaraan secara akumulatif sesuai flowchart pada modul:



Penjelasan Alur Flowchart:

• **1. Inisialisasi:** Inisialisasi variabel: `int jenis, durasi, total = 0;`.

• **2. Input Jenis:** Pengguna memasukkan kode jenis kendaraan (1 untuk Mobil, 2 untuk Motor, 0 untuk Keluar).

• **3. Pengecekan Kendaraan:** Diperiksa apakah `jenis == 1 || jenis == 2`. Jika YA (True), pengguna diminta menginputkan `durasi` parkir (jam). Selanjutnya jika `jenis == 1` (Mobil), total ditambah `durasi * 3000`. Jika bukan jenis 1 (Motor), total ditambah `durasi * 2000`. Jika TIDAK (False), langkah input durasi dan perhitungan dilewati.

• **4. Terminasi Loop:** Di akhir blok, diperiksa kondisi perulangan `while (jenis != 0)`. Jika jenis bukan 0, program berputar kembali ke langkah input jenis. Jika jenis sama dengan 0, perulangan berhenti.

• **5. Output:** Program menampilkan akumulasi nilai `total` pembayaran parkir ke layar konsol.

```java
import java.util.Scanner;

public class Parkir24 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int jenis, durasi, total = 0;

        do {
            System.out.print("Masukkan jenis kendaraan (1 Mobil, 2 Motor, 0 Keluar): ");
            jenis = sc.nextInt();

            if (jenis == 1 || jenis == 2) {
                System.out.print("Masukkan durasi parkir (jam): ");
                durasi = sc.nextInt();

                if (jenis == 1) {
                    total += durasi * 3000;
                } else {
                    total += durasi * 2000;
                }
            }
        } while (jenis != 0);

        System.out.println("Total pembayaran parkir: Rp " + total);
        sc.close();
    }
}

```